

■ Lab 2 — Resource Block : EC2 & Security Group

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Compris la structure d'un **resource block** Terraform
- Écrit votre premier fichier Terraform avec un bloc `terraform` et un bloc `provider`
- Déployé une **instance EC2** et un **Security Group sans règles** sur AWS
- Utilisé le workflow complet : `terraform init` → `validate` → `fmt` → `plan` → `apply` → `destroy`

■ Étape 1 — Rappel : Structure d'un Resource Block

Un resource block est composé de :

```
resource "<TYPE>" "<NOM_LOCAL>" {  
  attribut1 = "valeur1"  
  attribut2 = "valeur2"  
}
```

- **Mot-clé resource**
- **Type de ressource** : ex. `aws_instance`, `aws_security_group`
- **Nom local** : utilisé uniquement dans le code Terraform pour référencer la ressource
- **Attributs** : propriétés de la ressource

■ Terraform crée une ****dépendance implicite**** quand un attribut d'une ressource est référencé dans une autre. Pas besoin de `depends_on`` dans ce cas.

■ Étape 2 — Structure des fichiers

Placez-vous dans le dossier du lab :

```
cd labs/lab2-basics
```

Créez les fichiers suivants :

```
lab2-basics/  
■■■ versions.tf  
■■■ main.tf  
■■■ outputs.tf
```

■ Étape 3 — Bloc `terraform` et `provider`

`versions.tf`

```
terraform {  
  required_version = ">= 1.15.0"  
  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "eu-west-1"  
}
```

■ Le bloc `terraform` configure les comportements globaux : version minimale de Terraform et providers requis.

Le bloc `provider` configure l'authentification et la région AWS. Les credentials sont lus automatiquement depuis les variables d'environnement `AWS\_ACCESS\_KEY\_ID` et `AWS\_SECRET\_ACCESS\_KEY`.

■ Étape 4 — Security Group sans règles

`main.tf`

```
resource "aws_security_group" "lab2_sg" {  
  name      = "lab2-security-group"  
  description = "Security group sans regles - Lab 2"  
  
  tags = {  
    Name = "lab2-sg"  
    Lab  = "lab2"  
  }  
}
```

■ Ce Security Group est créé **sans aucune règle** ingress ou egress — c'est volontaire pour ce lab. On ajoutera des règles dans le Lab 4 avec les dynamic blocks.

■ Étape 5 — Instance EC2

Ajoutez dans `main.tf` :

```
resource "aws_instance" "lab2_instance" {
  ami           = "ami-0694d931cee176e7d"
  instance_type = "t2.micro"
  vpc_security_group_ids = [aws_security_group.lab2_sg.id]

  tags = {
    Name = "lab2-instance"
    Lab  = "lab2"
  }
}
```

■ `aws_security_group.lab2_sg.id` est une **dépendance implicite** — Terraform crée le Security Group avant l'instance automatiquement, sans qu'on ait besoin de le préciser.

■■ L'AMI `ami-0694d931cee176e7d` est une Amazon Linux 2023 valide en `eu-west-1`. Si vous utilisez une autre région, demandez l'AMI correspondante au formateur.

■ Étape 6 — Outputs

`outputs.tf`

```
output "instance_id" {
  description = "ID de l'instance EC2"
  value       = aws_instance.lab2_instance.id
}

output "instance_public_ip" {
  description = "IP publique de l'instance"
  value       = aws_instance.lab2_instance.public_ip
}

output "security_group_id" {
  description = "ID du Security Group"
  value       = aws_security_group.lab2_sg.id
}

output "security_group_name" {
  description = "Nom du Security Group"
  value       = aws_security_group.lab2_sg.name
}
```

■ Étape 7 — Workflow Terraform

1. Initialiser

```
terraform init
```

Résultat attendu : téléchargement du provider AWS dans `.terraform/`

2. Valider la configuration

```
terraform validate
```

Résultat attendu :

```
Success! The configuration is valid.
```

3. Formater le code

```
terraform fmt
```

■ ``terraform fmt`` corrige automatiquement l'indentation et l'alignement de votre code.

4. Planifier

```
terraform plan
```

■ Lisez attentivement le plan — il liste les **2 ressources** qui vont être créées : le Security Group et l'instance EC2.

5. Appliquer

```
terraform apply
```

Tapez `yes` pour confirmer. Attendez la fin du déploiement (~30 secondes).

6. Vérifier les outputs

```
terraform output
```

■ Étape 8 — Vérification sur AWS

```
# Vérifier l'instance EC2
aws ec2 describe-instances \
  --filters "Name=tag:Lab,Values=lab2" \
  --query "Reservations[*].Instances[*].[InstanceId,State.Name,PublicIpAddress]" \
  --output table

# Vérifier le Security Group
aws ec2 describe-security-groups \
  --filters "Name=tag:Lab,Values=lab2" \
  --query "SecurityGroups[*].[GroupId,GroupName]" \
  --output table
```

■ Étape 9 — Explorer les dépendances

Terraform peut générer un graphe de dépendances entre les ressources :

```
terraform graph
```

■ La sortie est au format DOT (Graphviz). On peut y voir clairement que ``aws_instance.lab2_instance`` dépend de ``aws_security_group.lab2_sg``.

■ Étape 10 — Nettoyage

```
terraform destroy
```

Tapez **yes** pour confirmer.

■■ Toujours détruire les ressources à la fin d'un lab pour éviter des coûts AWS inutiles.

■ Validation du Lab

#	Critère	Validé
1	`terraform init` réussi, provider AWS téléchargé	■
2	`terraform validate` retourne "Success"	■

#	Critère	Validé
3	`terraform plan` affiche 2 ressources à créer	■
4	`terraform apply` crée l'instance EC2 et le SG	■
5	`terraform output` affiche l'instance ID, l'IP et le SG	■
6	Instance et SG visibles via `aws ec2 describe-*`	■
7	`terraform destroy` supprime les 2 ressources	■

■ ■ Résolution des problèmes courants

Problème	Cause	Solution
`Error: No valid credential sources`	Credentials AWS manquants	Relancer `export AWS_ACCESS_KEY_ID=...`
`Error: InvalidAMIID`	AMI non disponible dans la région	Vérifier que la région est `eu-west-1`
`Error: UnauthorizedOperation`	Droits IAM insuffisants	Contacter le formateur
`Error: provider not found`	`terraform init` non exécuté	Relancer `terraform init`

■ ****Fin du Lab 2 — Vous avez déployé votre première infrastructure avec Terraform !****

Le Lab 3 explore la gestion du ****State file**** : `terraform state list`, `show`, `mv` et `refresh`.